

From Structural Invariants to a Red-Team Platform: Attack-Graph Mathematics for Autonomous Offensive Operations

VALEN Project

Working draft — engineering and methods report

Abstract

We describe the extension of VALEN — a neuro-symbolic structural-invariant verifier — into a full red-team *platform*: a multi-user team server that coordinates autonomous offensive operations across the kill chain (reconnaissance, Active Directory attack-graph analysis, command & control, credential recovery, phishing and exfiltration), under an explicit authorization and human-in-the-loop gating model. The core contribution is a layered attack-graph *mathematics*: structural signals (spectral, persistent and directed homology, discrete curvature), *flow* (Menger min-cut chokepoints, min-cost flow, articulation points), *probability* (absorbing Markov-chain hitting probabilities, solved exactly by $(I - Q)^{-1}R$), *formal synthesis* (Z3 Optimize bounded model checking that yields minimal-cost ordered exploit plans with AND-preconditions), and a *probabilistic context-free grammar* for password guessing. We report the system as deployed against OWASP crAPI (18/18 challenges solved autonomously, including three LLM prompt-injection challenges) and the Active Directory toy corpus, and we state precisely what each mathematical layer answers that a naive shortest-path or reachability query cannot.

1 Introduction

Classic semantic analysis answers *is this sink reachable by untrusted data?*. Offensive operations ask harder, quantitative questions:

- Which principals are *obligatory* on every path to the target (the chokepoints)?
- What is the *cheapest* ordered sequence of actions, given a model of stealth and effort, not just the fewest-hop one?
- What is the *probability* that a given entry point reaches the crown-jewel target, when the attacker moves according to some policy?
- With a *budget* of k compromises, what are the k cheapest independent routes?
- Which passwords should a spray try first, to maximize hits before lockout?

This paper reports the machinery added to VALEN to answer these, and the platform that operationalizes it.

2 Platform architecture

VALEN is a single installable Python package (`pip install valen`) with an optional Rust numeric core and a pure-Python fallback. Its server is a multi-user *team server* (users with roles

viewer/operator/ admin, PBKDF2 sessions, per-route RBAC, Server-Sent Events for live collaboration) that persists engagements, runs, and evidence artifacts in SQLite.

The red-team surface spans the kill chain:

- **Recon / enumeration** — nmap, masscan, gobuster, ffuf, nuclei wrappers with stealth profiles and CVE/KEV/EPSS intelligence.
- **Active Directory** — BloodHound/SharpHound ingestion into a typed IR graph, tier-0 attack paths, Kerberoast/AS-REP selection, GPP cpassword decryption, and command builders.
- **C2 and payloads** — Sliver integration (gRPC-first, shell fallback), msfvenom/handler/HTA planners, phishing (GoPhish) and exfiltration planners.
- **Hybrid agent** — a deterministic kill-chain planner with an operator approval queue; intrusive actions require explicit `-authorize`.
- **Reporting** — HTML / Markdown / JSON / SARIF with CVSS 3.1/4.0 and CVE/KEV/EPSS enrichment.

Everything offensive is gated by a scope allow-list (loopback by default), an execution allow-list (`-allow-exec`), and per-operator roles.

3 Attack-graph mathematics

The IR is a typed directed multigraph $G = (V, E)$ over principals. We layer four families of methods over it.

3.1 Structural signals

The base layer reuses the existing kernels: the combinatorial Laplacian and its Fiedler vector/ λ_2 , persistent H_0/H_1 homology, GLMY directed path homology, Forman–Ollivier Ricci curvature, and betweenness/PageRank. To these we add spectral centralities — eigenvector, Katz, and HITS (hubs/authorities) — for targeting and pivot selection.

3.2 Flow: chokepoints and disjoint plans

Menger min-cut. Given entry set S and target set T , we compute the minimum vertex set whose removal separates them. We reduce to an s - t max flow on the vertex-split graph (each vertex v becomes $v_{\text{in}} \rightarrow v_{\text{out}}$ of capacity 1; sources/targets unbounded) and run Edmonds–Karp; the saturated split edges on the source side of the residual graph are the *obligatory chokepoints*. A flow of magnitude $\geq \infty$ means a source is directly adjacent to a target (min-cut 0).

Min-cost flow. With a per-edge cost model (stealth + effort; e.g. MemberOf = 1, DCSync = 5), we send k units of flow at minimum cost via successive shortest augmenting paths (Bellman–Ford on the residual graph). Because internal vertices have capacity 1, the result is a set of *vertex-disjoint* cheapest attack routes — “with k compromises, the k cheapest independent ways in”.

Articulation points and bridges (Tarjan, on the symmetrized graph) rank the most disruptive principals and links: removing them fragments the network. The directed min-cut is the *defender/hardening* view; the undirected articulation points are the *attacker/disruption* view.

3.3 Probability: hitting time

Model the attack graph as an absorbing Markov chain: targets are absorbing, and each transient node transitions uniformly over its out-neighbours. The hitting probability $h(u)$ satisfies

$$h(u) = \mathbb{K}[u \in T] + \mathbb{K}[u \notin T] \cdot \frac{1}{\deg^+(u)} \sum_{v: u \rightarrow v} h(v),$$

computed either by value iteration or *exactly* by Gaussian elimination on $(I - Q)h = r$, where Q is the transient-to-transient transition and r the one-step mass into targets. This yields a probabilistic risk ranking, distinct from boolean reachability: in the toy corpus, an entry with three out-edges and a single route to the target scores $\frac{1}{3}$, while a direct entry scores 1.

3.4 Formal synthesis: minimal-cost exploit plans

Boolean reachability (Z3) answers *is it reachable?*. We upgrade to *synthesis*: model the graph as a transition system

$$\text{has}(s+1, n) \equiv \text{has}(s, n) \vee \bigvee_{e: m \rightarrow n} (\text{taken}_e \wedge \text{has}(s, m)),$$

with entries compromised at step 0, and ask `z3.Optimize` to minimize $\sum_e \text{taken}_e \cdot \text{cost}(e)$ subject to reaching the target within k steps. Reconstructing the ordered action sequence gives the technique-by-technique narrative. We extend this to **AND-preconditions**: compound actions $\{\text{sources} \rightarrow \text{target}\}$ that fire only when *all* sources are compromised (e.g. DCSync needs Domain Admins *and* a session on a Domain Controller).

3.5 Password guessing: PCFG

We implement Weir’s probabilistic context-free grammar: tokenize passwords into $L/D/S$ runs; the base structure (e.g. $L_6D_4S_1$) is the non-terminal and terminals are keyed by (class, length) — so no cross-length mixing (`summer+123` is rejected when only L_6D_4 was seen). The probability of a guess is $P(\text{structure}) \cdot \prod P(\text{terminal})$, and guesses are emitted in strictly decreasing probability by a best-first (Dijkstra) walk over the sorted per-slot terminals. Training corpus is a cracked hashcat potfile; an empty corpus falls back to a seed list.

4 Evaluation

4.1 Autonomous pentest against OWASP crAPI

Against a fresh OWASP crAPI 1.1.5 lab, the autonomous agent solves **18/18** documented challenges, including BOLA/BFLA, mass assignment, SSRF, NoSQL/SQL injection, JWT forgery, and three LLM prompt-injection challenges (routed through a local LiteLLM proxy). The BOLA detector reaches recall 1.0 / precision 0.90.

4.2 Active Directory toy corpus

On a synthetic SharpHound corpus the layers compose:

- *Chokepoints*: with entry BOB, min-cut 1 at the intermediate group IT; with entry ALICE (directly GenericAll on Domain Admins), min-cut 0.

- *Hitting*: ALICE = 1.0, BOB = $\frac{1}{3}$ (three out-edges, one to the target).
- *Synthesis*: ALICE $\xrightarrow{\text{GenericAll/T1098}}$ Domain Admins (cost 3.0), and ALICE $\xrightarrow{\text{MemberOf}}$ IT $\xrightarrow{\text{DCSync/T1003.006}}$ domain (cost 6.0); BOB is certified *unreachable* to Domain Admins.

5 Limitations and ethics

The platform is an *authorized-engagement* tool: scope is loopback by default, execution is behind `-allow-exec`, intrusive actions require operator approval, and every module carries a "for authorized engagements only" notice. The pure-Python numerical fallback is an honest approximation of the Rust core (persistent homology reduced to Betti numbers, GLMY to a cycle-rank estimate); set `VALEN_CORE_BIN` to use the compiled core. The PCFG prior is only as good as its training corpus; production use should feed it a real cracked corpus.

6 Conclusion

We turned a structural-invariant verifier into a red-team platform whose differentiator is *quantitative attack-graph mathematics*: chokepoints, least-cost and disjoint plans, exact hitting probabilities, formal minimal-cost exploit synthesis with AND-preconditions, and a PCFG password prior. Each layer answers a question that reachability and shortest paths cannot, and the whole is operationalized behind an explicit authorization model. All code is installable, tested (383 tests), and documented.